1. A mapper between URLs and view code
2. An abstraction for interacting with a database
3. A templating system to manage your HTML-like code

Of course, Django is way more than just three parts, but all the other parts are built using these three concepts. Django includes tools like a web interface for modifying content in the database and the code needed to write multiuser sites. Each of these is built on those same three ideas. This means once you've learned the key concepts, you can incorporate the tools into your site and customize them using the same knowledge you use to write your web application.

The first of the three ideas is about how code gets invoked when a user visits a web page. When a browser hits a URL, a web server is responsible for replying. Django isn't a web server; it is a framework called by a web server. The web server is configured to map certain URLs to certain content, and in the case of Django, some of the URLs get passed to the framework. This handoff is completed through a generic Python web interface called the *Web Server Gateway Interface* (WSGI).

When you visit a Django URL, the web server calls the framework through WSGI, at which point Django is responsible for generating the page response. The first core part of Django determines what code to call based on the URL. This is done through a mapping in your Django project. The mapping is between URLs and response handlers, known as *views*. A view is a function or class that generates content, which Django then parcels as a response to the web browser. You'll write your first view in chapter 2, shortly after you've created your first Django project.

The complete request flow from browser to response is shown in figure 1.1. It starts with the browser interacting with the web server, the web server calling Django through WSGI, Django mapping the visited URL to a view, and the view formulating a response.

The mapping between URLs and views is accomplished through a list data structure inside a Python file. Each mapping gets defined through the construction of a `path` object, which contains the URL to map and a reference to a view function. Mappings can be given names, so you can reference them elsewhere in your code; you use the name in your code, or in your HTML, and Django replaces it for you. This way, changing a URL only needs to be done in one place.

Visiting a URL causes Django to loop through all its mappings and look for a match. If a match is found, the corresponding view gets invoked. If no match is found, Django shows a 404 error page.

The vast majority of code you write for a Django project is the actual view code. Views return a response object that encapsulates the information to send back to the browser. The most common of these objects is `HttpResponse`, which returns text over HTTP, the body of which is typically HTML.

Inside your view, you determine what your visitor sees on the web page. This may involve interacting with a database. Your database might contain the inventory in your warehouse, a list of the books in your library, or the names of users who can access